Lab 03 Introduction to Java programming language

Task 01_Problem: Read an integer `N`, then read `N` integers into an array. Compute and print:

- min, max
- sum
- average (double, 2 decimals)
- count of even numbers

Input:
`N` then `N` integers.
Output:
Min, max, sum, average, even count (clearly labeled).
Constraints:
`1 ≤ N ≤ 10,000`.

Task 02_Problem:
Read `N` and `N` integers. Read an integer `k`. Produce a new array that is the original array **shifted right by k positions** (wrap-around).

Example: `[10, 20, 30, 40, 50]`, `k=2` → `[40, 50, 10, 20, 30]`.
Input: `N`, then `N` integers, then `k`.
Output: The shifted array on one line.

Task 03_Problem: Write a function:

```
static int countOccurrences(int[] a, int target)
```

It returns how many times `target` appears in array `a`.
In `main`, read `N`, array values, then a `target`. Call the function and print the result.
Input: `N`, `N` integers, `target`.
Output: `target appears X times`

Task 05_Problem: Read `R` and `C`, then read an `R × C` table of integers into `int[][]`. Print:

- sum of each row (one per line)
- sum of each column (one per line)
- total sum of all values

Input: `R C` then `R*C` integers.
Output: Row sums, column sums, total sum.
Constraints: `1 ≤ R, C ≤ 50`.

Task 06_Scenario:
A classroom has `R` rows and `C` seats per row. `0` means available, `1` means taken.
You need to find the **first row** (smallest row index) that contains a **consecutive block of K available seats**.
Problem: Read `R`, `C`, `K`, then the seating matrix. Output:

- `Row = <rowIndex>, StartSeat = <colIndex>` for the first valid block, or
- `No block found`

Dr. Oussama SERHANE (o.serhane@esi-sba.dz)

Input: `R C K` then `R*C` values (0/1).
Output: Location of the first available block or not found.
Constraints:

- `1 ≤ K ≤ C`
- Prefer earliest row, then earliest start column.

Task 07_Problem: Implement these functions and use them in `main`:

```
static int maxValue(int[][] m)
static int sumMainDiagonal(int[][] m)   // only if square
static boolean isSquare(int[][] m)
```

In `main`, read a matrix and:

- print whether it is square
- print max value
- if square, print diagonal sum; otherwise print "No diagonal sum".

Input: `R C` then `R*C` integers.
Output: Square? Max? Diagonal sum (if applicable).
Constraints: None beyond valid sizes.
Hints: For diagonal sum, add `m[i][i]` for `i = 0..R-1` (only when `R == C`).

Task 08_Problem: Write **two recursive functions**:

1. `sumDigits(n)` returns the sum of digits of a non-negative integer `n`.
   Example: `sumDigits(5029) = 16`.
2. `reverseNumber(n)` returns the digits of `n` reversed.
   Example: `reverseNumber(5029) = 9205`.

In `main`, read an integer `n`, call both functions, print results.
Input: One non-negative integer `n`.
Output: Sum of digits and reversed number.
Constraints:

- Must be recursive (no loops inside these functions).
- Handle `n = 0` correctly.